

Axi Command Palette (Axi_Beta) | Technical, Architectural & Usage Documentation

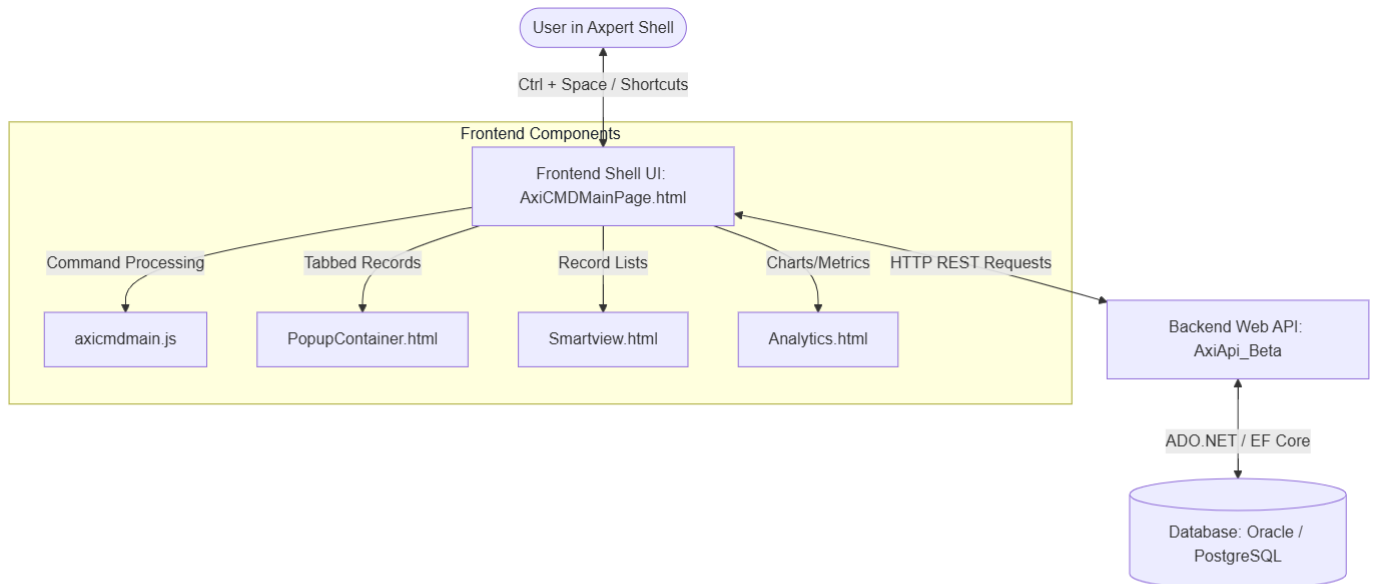
Welcome to the comprehensive documentation for the **Axi Command Palette** plugin (`Axi_Beta`). This document provides system administrators, database administrators, developers, and end-users with a complete guide to the system's architecture, database schema, APIs, frontend layouts, keyboard shortcuts, installation procedures, and daily usage patterns.

Table of Contents

- System Architecture Overview
- Database Schema & Objects
 - Tables Catalog
 - Stored Procedures & Functions (PostgreSQL vs. Oracle)
- Backend API Specifications (AxiApi_Beta)
- Frontend Components & User Interface
- Interactive Keyboard Shortcuts
- Supported Command Catalog
- Installation & Deployment Steps
- User Usage Guide
 - Activating the Palette
 - Constructing Commands with Auto-Complete Hints
 - Pinning Favourites and Recents
 - Tabbed Window Management (Popup Container)
 - Interactive Lists (Smartview)
 - Visual Dashboards (Analytics)
- Developer/Administrator Configuration Guide
 - Adding Custom Commands
 - Registering Auto-Complete Prompts
 - Direct SQL Metadata Generation
- Troubleshooting & Maintenance

System Architecture Overview

The Axi Command Palette is designed as a modular, high-performance command-driven search and navigation utility that integrates directly into the Aexpert Web Shell interface. The architecture is split into three main layers:



1. Frontend Layer

- **AxiCMDMainPage.html** (AxiCMDMainPage.html): Integrates directly within Aexpert as the master template.
- **axicmdmain.js** (axicmdmain.js): Contains the main autocomplete suggestion logic, command parser, state machine, and API client.
- **Smartview.html** (Smartview.html): Handles interactive record list views, filtering, utility actions, and multi-select deletion.
- **PopupContainer.html** (PopupContainer.html): Provides a tabbed interface to manage multiple open transactions in parallel.
- **Analytics.html** (Analytics.html): Coordinates dynamic entity analysis, rendering chart panels, custom KPI metrics, and layout scaling.

2. Backend API Layer (AxiApi_Beta)

- Built on **.NET 8.0** for high performance and native async request processing.
- Acts as a microservice running under IIS (or stand-alone).
- Resolves database connection settings from the shared `appsettings.ini` file in the parent Arm microservices directory.
- Supports both **PostgreSQL** (`Npgsql`) and **Oracle** (`Oracle.ManagedDataAccess`) data providers.

3. Database Layer

- Maintains configurations for commands and dynamic parameter prompts.
- Implements security and permission queries to filter search suggestions based on the logged-in user's roles and responsibilities.

Database Schema & Objects

The database schema scripts are located under the Structures directory.

Tables Catalog

The following tables define the command structure and properties:

Table Name	Description
<code>axi_commands</code>	Stores root command words and their groupings.
<code>axi_commands_prompts</code>	Stores prompt inputs, order, dynamic sources, and URL parameters for each token position.
<code>axp_tstruct_props</code>	Manages additional properties for Tstruct definitions (e.g., custom primary key fields).
<code>axdirectsql_metadata</code>	Caches column names and metadata details for direct SQL queries (<code>ADS</code>).

Stored Procedures & Functions (PostgreSQL vs. Oracle)

Depending on the database engine, the structures differ slightly to accommodate platform features:

- **PostgreSQL Scripts:** Structures/Postgre/Scripts/
- **Oracle Scripts:** Structures/Oracle/Scripts/

Core Functions:

- `fn_axi_getstructures_meta`
 - **PostgreSQL:** `fn_axi_getstructures_meta`
 -

- **Oracle:** Returns a pipelined cursor table type `AXI_GETSTRUCTURES_META_tbl` .
- **Purpose:** Aggregates all accessible Tstructs, lviews, Pages, and Axpert Data Sources (ADS) for the logged-in user. Filters by user responsibility permissions.

`fn_axi_getstructs_obj`

- **PostgreSQL:** `fn_axi_getstructs_obj`
- **Oracle:** Returns a pipelined type `axi_getstructs_obj_tbl` .
- **Purpose:** Fetches records from a target Tstruct base table dynamically based on a search value. Evaluates row-level dimension filters and user read-write permissions.

`fn_permissions_getpermission`

- **PostgreSQL:** `fn_permissions_getpermission`
- **Purpose:** Validates whether the logged-in user is authorized to perform Create, View, or Edit operations on a specific transaction (transid).

`fn_axi_getkeyvalueswithfieldnameslist`

- **PostgreSQL:** `fn_axi_getkeyvalueswithfieldnameslist`
- **Purpose:** Dynamically extracts the primary key values and available column descriptors for a given transaction structure.

`axi_firesql_v2`

- **PostgreSQL:** `axi_firesql_v2`
- **Purpose:** A secure SQL executor that allows parametrized queries to prevent SQL injection vulnerabilities.

[!NOTE]

In Oracle environments, SQL procedures return pipelined table sets using custom types like `AXI_GETSTRUCTURES_META` and `axi_getstructs_obj` to achieve maximum compatibility with the .NET Entity Framework Core mapping.

Backend API Specifications (AxiApi_Beta)

The .NET 8 backend API acts as the bridge. It provides two key endpoints consumed by the frontend javascript client.

1. Retrieve Configured Objects & Permissions

- **Route:** `/api/v1/Axi/axi_get`
- **Method:** `POST`
- **Headers:** `Content-Type: application/json`
- **Query Parameters / Body:** Passes parameters such as `username` , `roles` , `responsibility` , and `mode` .
- **Response:** A JSON collection of structs, pages, and direct SQL datasets mapped to the user's workspace profile.

2. Manage Favorites / Recents

-

- **Route:** `/api/v1/Axi/user-favourites`
- **Method:** `GET / POST / DELETE`
- **Purpose:** Fetches, adds, or deletes commands frequently executed by the user. Caches these actions locally to populate the initial autocomplete list on `Ctrl + Space`.

Frontend Components & User Interface

The frontend code leverages existing Apxert UI style libraries and custom CSS extensions:

AxiCMDMainPage.html (AxiCMDMainPage.html):

- Hosts the search container `<div class="AXI-Sec">` which floats at the top of the interface.
- Defines modal overlays for confirming deletion of favorites, custom alerts, and dynamic loading states.

axicmdmain.js (axicmdmain.js):

- Maintains autocomplete logic. It tokenizes user inputs and maps them to the matching command group (e.g., `Create`, `Edit`, `View`, `DevTools`).
- Appends dynamic parameter helpers in `axiHint` to instruct users on what parameters (like field values or search strings) are expected next.

axicmdmain.css (axicmdmain.css):

- Provides clean aesthetics: rounded boundaries, transparent inputs, custom action overlays, and scrollable recommendation list containers.

Interactive Keyboard Shortcuts

Users can operate the command palette entirely via key combinations:

Keyboard Shortcut	Context / Input Focus	Action Triggered
<code>Ctrl + Space</code>	Anywhere in the application	Toggle Command Palette (opens or focuses the search input)
<code>Ctrl + Enter</code>	Inside <code>Axi-Searchinp</code>	Run / Execute Command (triggers navigation, form submission, etc.)
<code>Ctrl + S</code>	Inside active forms	Save Record (automatically commits active structural data)
<code>Ctrl + Shift + Enter</code>	Selecting suggestions	Open in Popup (loads the screen within the tabbed iframe).
<code>Ctrl + Alt + Enter</code>	Suggestions selected	View Command Source / Metadata (opens underlying data or documentation)
<code>Backspace</code>	After command word	Command Reset (erases trailing tokens and parameters)

Supported Command Catalog

Axi evaluates the first word of the user's input to determine the command state.

1. Create [tstruct]

- **Description:** Opens a fresh form to create a new transaction record.
- **Example:** `Create customer`

2. Edit [tstruct] [search value] [object name] with [field name]

- **Description:** Navigates directly to an existing transaction record to edit specific fields.
- **Example:** `Edit customer 10452 John Doe with Email`

3. View [Tstruct/Iview/Page/ADS] [search value] [object name]

- **Description:** Displays records in read-only view. Works with transaction forms (`Tstruct`), database views (`Iview`), standard web pages (`Page`), or custom direct SQL configurations (`ADS`).
- **Example:** `View invoice 2099`

4. Configure [type] [name] [key field]

- **Description:** Quick access to system configuration settings. Supported types include `Rule` , `KeyField` , `User` , `Role` , `Responsibility` , `Dimension` , `Form Notification` , and `PEG` .
- **Example:** `Configure User admin`

5. Upload & Download

- **Description:** Import or export datasets.

6. DevTools [type] [name]

- **Description:** Opens utility tools. Types include `DB Explorer` , `App Variables` , `Arrange Menu` , `Dev Option` , `Language` , and `Custom Plugin` .
- **Example:** `DevTools DB Explorer`

7. Analyse [entity]

- **Description:** Displays the custom dashboard (`Analytics.html`) showing charts, summary metrics, and KPI graphs for the selected entity.
 - **Example:** `Analyse sales_ledger`
-

Installation & Deployment Steps

Follow this step-by-step pipeline to host and configure the plugin:

Step 1: Core Package Deployment

Open the **AxInstaller** tool.

Install the **Axi_Beta** plugin.

Ensure source files populate correctly under `D:\Axpert11.4\AxpertWebLatest\AxpertPlugins\Axi_Beta\`.

Step 2: Front-End Registration

Copy `AxiCMDMainPage.html` from `HTMLPages/` directory.

Paste it into the `D:\Axpert11.4\AxpertWebLatest\CustomPages\` folder.

Warning: Do not change the filename, as routing relies on `AxiCMDMainPage.html`.

Step 3: Enable Template inside Axpert Options

Log in to the **AxpertWeb** interface as administrator.

Navigate to **Dev Options** -> **Application Template**.

Choose `AxiCMDMainPage.html` from the dropdown list.

If missing from the dropdown: Go to **Configuration Property List** -> edit **Application Template** -> add `AxiCMDMainPage.html` to the Values collection.

Step 4: IIS Backend Setup (`AxiApi_Beta`)

Copy the `AxiApi_Beta` folder.

Paste it into the **Arm microservices** publication directory on your target IIS server.

Configure a new Application Pool in IIS Manager:

- **Name:** `AxiApi_Beta`
- **.NET CLR Version:** `No Managed Code` (since it uses .NET 8 runtime directly).

Create an application pointing to the `AxiApi_Beta` folder.

Copy `appsettings.ini` from `../AxpertWebScript/` and place it in the **Arm microservices** parent directory so the API can read connection strings.

Assign **Read & Write** folder permissions to the App Pool Identity (e.g. `IIS_IUSRS`).

Step 5: Database Scripts Execution

Execute the appropriate scripts based on your target database:

- **PostgreSQL:** Run the scripts inside `Structures/Postgre/Scripts/`:

```
axi_axdirectsql_tables.sql
```



```
axi_command_tables.sql
axi_dependent_tables.sql
axi_functions.sql
```

- **Oracle:** Run the scripts inside Structures/Oracle/Scripts/:

```
axi_axdirectsql_tables.sql
axi_command_tables.sql
axi_dependent_tables.sql
axi_functions.sql
```

User Usage Guide

Activating the Palette

Focus anywhere on the Axpert application home shell.

Press **Ctrl + Space** on your keyboard. The search input bar labeled "Axpert AI" will glide into focus, overlaying suggestions.

To hide the palette, click anywhere outside the suggestion box or press **Escape** .

Constructing Commands with Auto-Complete Hints

Axi features a **predictive parameter builder** that dynamically shows context hints inside the search bar as you type:

- 1 Step 1: Type "Create" —————> Suggestion lists Tstructs (e.g., "customer")
- 2 Step 2: Type "customer" —————> Input hint shows "[tstruct name]"
- 3 Step 3: Press "Ctrl + Enter" —> Opens Customer Creation form immediately.

If editing a record:

Type **Edit** .

Type **customer** .

Type a search value (like a customer ID **10452**). Axi will fetch matching records from the database. Select **John Doe** from the recommendations list.

Type **with** .

Type **Email** . Axi will position the cursor directly onto the Email field of John Doe's customer record for immediate editing.

Pinning Favourites and Recents

Frequently used commands can be saved for quick execution:

Type any valid command syntax into the input bar and execute it(e.g., `View invoice 2099`).

Click the **Bookmark icon** (`axiFavouriteBtn`) located on the right side of the input.

A popup modal will appear prompting you to name this favorite (e.g., "Check Invoice 2099").

Type the name and press **Enter** (or click **Save**).

Next time you open the Command Palette with `ctrl + space` , your saved favorites will populate immediately at the top of the suggestion panel.

Removing a Favorite: Hover over the favorite command card in the suggestion list and click the **X icon** (Delete) to remove it from the backend list.

Tabbed Window Management (Popup Container)

When launching screens from the Command Palette, you can preserve your current shell layout by loading them into a tabbed popup:

Type your command (e.g., `Create customer`).

Press `ctrl + Shift + Enter` (instead of standard `ctrl + Enter`).

The screen will load in a tab inside the **Popup Container**.

You can open multiple transactions sequentially. Each will open as a separate tab (e.g., "Customer Create", "Invoice 2099", "Sales Chart").

Click on the tabs to switch views instantly. Click the **Close (X) button** on each tab to release the resources.

Interactive Lists (Smartview)

When searching lists of entities, Axi routes the data into the interactive **Smartview** panel:

- **Search and Filter:** Use the search bar in the Smartview toolbar to search records, or click the **Tune** icon to open advanced query filters.
- **Column Customization:** Click the **Services** icon (Utilities) and select **Select Fields**. Check or uncheck columns to customize your data table view.
- **Batch Action (Delete):** Select multiple records using checkboxes, then click the **Trash/Delete** icon in the toolbar to batch-delete them.

Visual Dashboards (Analytics)

Launch the analytics command to monitor business operations:

Type `Analyse` followed by your data ledger name (e.g., `Analyse sales_ledger`).

A visual dashboard will load, rendering bar, pie, and line charts based on real-time database queries.

Adjust Layout: Click the `+` (Plus) or `-` (Minus) icons in the Analytics toolbar to scale the size of cards, modifying how many charts appear per row.

Data Refresh: Click the **Refresh** icon to query the database and reload the charts.

🔧 Developer/Administrator Configuration Guide

Administrators can customize the commands, parameters, and database bindings.

Adding Custom Commands

To add a new command group:

Assign a unique `cmdtoken` identifier.

Insert a row into `axi_commands` :

```
1  INSERT INTO axi_commands (cmdtoken, command_group, command, active)
2  VALUES (12, 'Publish', '', 'T');
```

Registering Auto-Complete Prompts

To specify what parameter recommendations are shown when the command is typed, insert details into

`axi_command_prompts` :

- `cmdtoken` : Maps to the command group.
- `wordpos` : The word index location in the typed command where this autocomplete is triggered (1-indexed).
- `prompt` : Context helper text (e.g., "object name").
- `promptsource` : The name of the database table, database function, or static list values to load suggestions from.
- `extraparams` : Parameter mapping bindings (e.g., passing `:username` , `:userrole` , `:transid`).

Example for adding a prompt parameter at position 2:

```
1  INSERT INTO axi_command_prompts (id, cmdtoken, wordpos, prompt, promptsource, extraparams)
2  VALUES (gen_random_uuid(), 12, 2, 'API Name', 'axi_publishapi', ':username');
```

🔧 Troubleshooting & Maintenance

- **Command Palette is not opening:**
 - Ensure the web application has registered `AxiCMDMainPage.html` as the Application Template in Dev Options.
 - Check if the console logs show `Axi Input not ready yet... waiting` . If so, ensure that the search input ID (`Axi-Searchinp`) is present in the rendered HTML.
- **Search suggestions are empty:**
 - Verify the backend API is running. Open the IIS application URL in your browser (e.g., `http://<server>/AxiApi_Beta/api/v1/Axi/axi_get`) and verify it responds.

- ~ Check the API's log file located in `AxiApi_Beta/logs/log.txt` for database connection errors.
- ° Verify that you executed the database scripts (specifically `axi_commands` and `axi_command_prompts`) and the tables are populated.

- **Data permissions / Row filtering issue:**

- ° Axi evaluates permissions via `fn_permissions_getpermission`. Verify that the user's role responsibilities are correctly assigned in Axpert's standard user settings pages.